Git 版本控制教程：督办管理系统协作指南（新手友好版）

目录

1. Git 简介
2. Git 安装
3. 项目克隆
4. Git 基本操作
5. 分支管理
6. 协作流程
7. Git 规范
8. 常见问题与解决方案
9. 终端操作指南
10. 详细实战演练

Git 简介

Git 是一个分布式版本控制系统，用于跟踪项目中文件的变化。简单来说，它就像是一个时光机，可以记录你对项目的每一次修改，让你可以随时回到过去的任何一个版本。

为什么使用 Git?

- 可以记录项目的历史变更，知道谁在什么时候做了什么修改
- 允许多人同时协作开发，不会互相干扰
- 支持分支管理，方便并行开发多个功能
- 可以回滚到任意历史版本，当出现问题时可以快速恢复
- 提供冲突解决机制，当多人修改同一文件时可以和平解决

Git 安装

Windows 系统

1. **访问 Git 官网**：打开浏览器，输入 https://git-scm.com/downloads
2. **下载安装包**：点击 "Windows" 下载链接

3. **运行安装程序**：双击下载的 .exe 文件，按照默认选项一步步点击 "Next" 即可
4. **验证安装**：安装完成后，按下 `Win + R`，输入 `cmd`，回车，在命令提示符中输入 `git --version`，如果显示版本号，则安装成功

macOS 系统

1. **打开终端**：在 Launchpad 中找到 "终端" 应用并打开
2. **安装命令行工具**：输入 `xcode-select --install`，按照提示完成安装
3. **验证安装**：输入 `git --version`，如果显示版本号，则安装成功

Linux 系统

1. **打开终端**：按下 `Ctrl + Alt + T`
2. **使用包管理器安装**：
   - Ubuntu/Debian: 输入 `sudo apt install git`，回车，输入密码
   - CentOS/RHEL: 输入 `sudo yum install git`，回车，输入密码
3. **验证安装**：输入 `git --version`，如果显示版本号，则安装成功

项目克隆

步骤 1：获取项目地址

项目地址：`https://github.com/Bob-haha/supervision-v4.6.git`

步骤 2：克隆项目（详细步骤）

Windows 系统

1. **打开命令提示符**：按下 `Win + R`，输入 `cmd`，回车
2. **导航到存储目录**：

```
# 例如，导航到 D 盘的 Code 目录
D:
cd Code
```

3. **克隆项目**：

```
git clone https://github.com/Bob-haha/supervision-v4.6.git
```

4. **进入项目目录**：

```
cd supervision-v4.6
```

5. **验证**：输入 `dir`，可以看到项目文件

macOS/Linux 系统

1. **打开终端**
2. **导航到存储目录**：

```
# 例如，导航到 Documents 目录
cd ~/Documents
```

3. **克隆项目**：

```
git clone https://github.com/Bob-haha/supervision-v4.6.git
```

4. **进入项目目录**：

```
cd supervision-v4.6
```

5. **验证**：输入 `ls`，可以看到项目文件

Git 基本操作

1. 查看状态

```
git status
```

示例输出：

```
on branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)
```

```
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
```

这个命令会显示当前工作目录的状态，包括已修改、已暂存和未跟踪的文件。

2. 添加文件到暂存区

```
# 添加单个文件
git add README.md

# 添加所有文件
git add .
```

示例：

- 如果你修改了 [README.md](#) 和 src/App.vue，想只提交 [README.md](#)：

  ```
  git add README.md
  ```

- 如果你修改了多个文件，想全部提交：

  ```
  git add .
  ```

3. 提交更改

```
git commit -m "提交信息"
```

示例：

- 修复了一个 bug：

  ```
  git commit -m "fix: 修复下发功能参数传递错误"
  ```

- 添加了一个新功能：

```
git commit -m "feat: 添加任务进度跟踪功能"
```

- 更新了文档：

```
git commit -m "docs: 更新 README.md 说明"
```

4. 推送到远程仓库

```
git push origin 分支名
```

示例：

- 推送 main 分支：

```
git push origin main
```

- 推送功能分支：

```
git push origin feature/task-tracking
```

5. 从远程仓库拉取更新

```
git pull origin 分支名
```

示例：

- 拉取 main 分支的最新代码：

```
git pull origin main
```

分支管理

1. 查看分支

```
git branch
```

示例输出：

```
* main
  feature/task-tracking
  bugfix/login-issue
```

带 `*` 的分支是当前所在的分支。

2. 创建分支

```
git checkout -b 分支名
```

示例：

- 创建一个新的功能分支：

  ```
  git checkout -b feature/dashboard-chart
  ```

- 创建一个 bug 修复分支：

  ```
  git checkout -b bugfix/sync-issue
  ```

3. 切换分支

```
git checkout 分支名
```

示例：

- 切换到 main 分支：

  ```
  git checkout main
  ```

- 切换到功能分支：

  ```
  git checkout feature/dashboard-chart
  ```

4. 合并分支

```
# 切换到目标分支
git checkout 目标分支
# 合并源分支
git merge 源分支
```

示例:

- 将功能分支合并到 main 分支:

```
git checkout main
git merge feature/dashboard-chart
```

5. 删除分支

```
# 删除本地分支
git branch -d 分支名

# 删除远程分支
git push origin --delete 分支名
```

示例:

- 删除本地的功能分支:

```
git branch -d feature/dashboard-chart
```

- 删除远程的功能分支:

```
git push origin --delete feature/dashboard-chart
```

协作流程

1. 标准工作流程（详细步骤）

1. **拉取最新代码**:

```
git pull origin main
```

2. **创建功能分支**：

```
git checkout -b feature/功能名称
```

3. **进行开发**：修改代码，例如修改 src/views/Task/index.vue 文件

4. **添加并提交**：

```
git add .
git commit -m "feat: 添加任务筛选功能"
```

5. **推送分支**：

```
git push origin feature/功能名称
```

6. **创建 Pull Request**：

   - 打开 GitHub 上的项目页面
   - 点击 "Pull requests" 选项卡
   - 点击 "New pull request" 按钮
   - 选择你的功能分支和目标分支（通常是 main）
   - 填写 PR 标题和描述，点击 "Create pull request" 按钮

7. **代码审查**：团队成员会审查你的代码，可能会提出修改建议

8. **合并分支**：审查通过后，项目维护者会合并你的分支到 main 分支

9. **删除分支**：合并后，你可以删除功能分支

```
git branch -d feature/功能名称
git push origin --delete feature/功能名称
```

2. 处理冲突（详细步骤）

当多人修改同一文件时，可能会产生冲突。例如，你和同事都修改了 src/App.vue 文件的同一部分。

解决冲突的步骤：

1. **拉取最新代码**：

```
git pull origin main
```

Git 会提示冲突，例如：

```
Auto-merging src/App.vue
CONFLICT (content): Merge conflict in src/App.vue
Automatic merge failed; fix conflicts and then commit the result.
```

2. **打开冲突文件**：使用编辑器打开 src/App.vue 文件，你会看到类似这样的内容：

```
<<<<<<<< HEAD
<div class="app-wrapper">
  <!-- 旧代码 -->
</div>
========
<div class="app-container">
  <!-- 新代码 -->
</div>
>>>>>>>> feature/功能名称
```

3. **手动编辑文件**：删除冲突标记（`<<<<<<<`, `=======`, `>>>>>>>`），保留你认为正确的代码，例如：

```
<div class="app-container">
  <!-- 新代码 -->
</div>
```

4. **添加解决冲突后的文件**：

```
git add src/App.vue
```

5. **提交解决冲突**：

```
git commit -m "解决冲突：合并 App.vue 文件"
```

6. **推送更改**：

```
git push origin feature/功能名称
```

Git 规范

1. 分支命名规范

- **main**：主分支，用于发布生产版本，任何人都不能直接修改
- **develop**：开发分支，集成所有功能，由项目维护者管理
- **feature/功能名称**：功能分支，用于开发新功能，例如 `feature/dashboard-chart`
- **bugfix/问题描述**：修复分支，用于修复 bug，例如 `bugfix/login-issue`
- **hotfix/紧急修复**：紧急修复分支，用于生产环境的紧急修复，例如 `hotfix/server-down`

2. 提交信息规范

提交信息应遵循以下格式：

```
类型：简短描述

详细描述（可选）

关联 issue（可选）
```

类型包括：

- **feat**：新功能
- **fix**：修复 bug
- **docs**：文档更新
- **style**：代码风格调整
- **refactor**：代码重构
- **test**：测试相关
- **chore**：构建或依赖更新

好的提交信息示例：

```
fix：修复下发功能参数传递错误

修改 TaskFormDialog.vue 中的 open 方法，使其能够处理两个参数（新级别和父任务ID）
```

```
关联 #123
```

不好的提交信息示例：

- `fix bug`（太模糊）
- `修改了文件`（没有具体说明）
- `添加新功能`（没有说明是什么功能）

3. 代码审查规范

- 提交 PR 前，确保代码通过构建和测试
- PR 描述应清晰说明修改内容和原因
- 团队成员应及时审查 PR，提供建设性反馈
- 至少需要 1 个审查通过才能合并
- 合并前解决所有审查意见

常见问题与解决方案

1. 忘记提交信息

问题：提交时忘记写提交信息，Git 打开了编辑器

解决方案：

- 如果你还在编辑器中，直接输入提交信息，保存并退出
- 如果你已经退出，可以使用 `git commit --amend` 命令修改最后一次提交的信息

2. 推送被拒绝

问题：执行 `git push` 时，收到 "rejected" 错误

原因：远程仓库的代码比你的本地代码新

解决方案：

```
# 先拉取最新代码并重新基变
git pull origin 分支名 --rebase
# 解决冲突后推送
git push origin 分支名
```

3. 撤销本地修改

问题：修改了文件，但想恢复到修改前的状态

解决方案：

```
# 撤销未暂存的修改
git checkout .

# 撤销已暂存的修改
git reset HEAD .
git checkout .
```

4. 回滚到之前的版本

问题：提交了错误的代码，想回到之前的版本

解决方案：

```
# 查看历史记录，找到要回滚的版本
git log
# 回滚到指定版本
git reset --hard 提交哈希值
```

示例：

```
git log
# 输出类似：
# commit 1a2b3c4d5e6f7g8h9i0j (HEAD -> main)
# Author: Your Name <your.email@example.com>
# Date:   Today
#
#     fix: 修复登录问题
#
# commit 9i8h7g6f5e4d3c2b1a0j
# Author: Your Name <your.email@example.com>
# Date:   Yesterday
#
#     feat: 添加用户注册功能
```

```
# 回滚到添加用户注册功能的版本
git reset --hard 9i8h7g6f5e4d3c2b1a0j
```

终端操作指南

Windows 系统

方法 1：使用命令提示符

1. 按下 `Win + R` 键
2. 输入 `cmd`，回车
3. 在打开的命令提示符窗口中输入 Git 命令

示例：

```
C:\Users\YourName> D:
D:\> cd Code
D:\Code> git clone https://github.com/Bob-haha/supervision-v4.6.git
```

方法 2：使用 PowerShell

1. 按下 `Win + R` 键
2. 输入 `powershell`，回车
3. 在打开的 PowerShell 窗口中输入 Git 命令

示例：

```
PS C:\Users\YourName> D:
PS D:\> cd Code
PS D:\Code> git clone https://github.com/Bob-haha/supervision-v4.6.git
```

方法 3：使用 Git Bash

1. 安装 Git 时会自动安装 Git Bash
2. 在开始菜单中找到 Git Bash 并打开
3. 在 Git Bash 窗口中输入 Git 命令

示例：

```
$ cd /d/Code
$ git clone https://github.com/Bob-haha/supervision-v4.6.git
```

macOS 系统

1. 打开 Launchpad
2. 找到并点击 "终端"
3. 在终端窗口中输入 Git 命令

示例:

```
$ cd ~/Documents
$ git clone https://github.com/Bob-haha/supervision-v4.6.git
```

Linux 系统

1. 按下 `Ctrl + Alt + T` 键
2. 在打开的终端窗口中输入 Git 命令

示例:

```
$ cd ~/Documents
$ git clone https://github.com/Bob-haha/supervision-v4.6.git
```

在 Trae IDE 中使用终端

1. 打开 Trae IDE
2. 点击顶部菜单中的 "Terminal"
3. 选择 "New Terminal"
4. 在打开的终端窗口中输入 Git 命令

示例:

```
(TraeAI) E:\CODE\supervision-v4.6> git status
```

详细实战演练

演练 1：第一次克隆和提交

场景：你是第一次参与项目，需要克隆项目并提交你的第一个修改。

步骤：

1. **打开终端**（根据你的操作系统选择合适的方法）
2. **导航到存储目录**：

```
# Windows
D:
cd Code

# macOS/Linux
cd ~/Documents
```

3. **克隆项目**：

```
git clone https://github.com/Bob-haha/supervision-v4.6.git
```

4. **进入项目目录**：

```
cd supervision-v4.6
```

5. **查看当前分支**：

```
git branch
```

输出：`* main`

6. **创建功能分支**：

```
git checkout -b feature/add-readme-section
```

7. **修改文件**：使用编辑器打开 [README.md](#) 文件，添加一个新的部分
8. **查看状态**：

```
git status
```

输出：

```
On branch feature/add-readme-section
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
```

9. **添加文件**：

```
git add README.md
```

10. **提交更改**：

```
git commit -m "docs: 添加项目结构说明部分"
```

11. **推送分支**：

```
git push origin feature/add-readme-section
```

12. **在 GitHub 上创建 PR**：
    - 打开 GitHub 上的项目页面
    - 点击 "Pull requests" 选项卡
    - 点击 "New pull request" 按钮
    - 选择 `feature/add-readme-section` 分支和 `main` 分支
    - 填写 PR 标题和描述
    - 点击 "Create pull request" 按钮

演练 2：更新代码并解决冲突

场景：你正在开发一个功能，同时其他同事也在修改同一文件。

步骤：

1. **切换到 main 分支**：

```
git checkout main
```

2. **拉取最新代码**：

```
git pull origin main
```

3. **切换回功能分支**：

```
git checkout feature/my-feature
```

4. **合并 main 分支**：

```
git merge main
```

5. **解决冲突**：

   - Git 提示冲突，例如：`CONFLICT (content): Merge conflict in src/App.vue`
   - 打开 src/App.vue 文件，找到冲突部分
   - 编辑文件，删除冲突标记，保留正确的代码
   - 保存文件

6. **添加解决冲突后的文件**：

```
git add src/App.vue
```

7. **提交解决冲突**：

```
git commit -m "解决冲突：合并 App.vue 文件"
```

8. **推送更新**：

```
git push origin feature/my-feature
```

演练 3：撤销错误的修改

场景：你修改了一个文件，但发现修改有误，想恢复到原始状态。

步骤：

1. **查看状态**：

```
git status
```

输出：

```
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

        modified:   src/App.vue
```

2. **撤销修改**：

```
git checkout -- src/App.vue
```

3. **验证**：

```
git status
```

输出：

```
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

总结

Git 是一个强大的版本控制工具，掌握它对于团队协作至关重要。通过本教程，你应该已经了解了 Git 的基本概念、操作方法和协作流程。

学习 Git 的小技巧:

- 实践是学习 Git 的最好方法，多动手操作
- 遇到问题时不要 panic，参考本教程或 Git 官方文档
- 先从基本操作开始，逐步学习高级功能
- 养成良好的 Git 习惯，遵循团队的 Git 规范

记住，每个人都是从新手过来的，犯错误是正常的。随着使用次数的增加，你会越来越熟练地使用 Git。

祝大家协作愉快！